

扫码SDK接入文档

SDK初始化

最简单的情况下SDK初始化后，就可以使用扫码服务。目前提供两种初始化方式：

```
XcBarcodeScanner.init(Context context, ScannerResult scannerResult)
```

```
XcBarcodeScanner.init(Context context, ScannersSymResult scannersSymResult)
```

回调类：

```
public interface ScannerResult {  
    void onResult(String result); // 仅返回条码结果  
}  
  
public interface ScannersSymResult {  
    void onResult(String sym, String barCode); // 返回条码结果和条码类型  
}
```

扫码SDK初始化后，就和系统的扫码服务建立了连接，此时扫码结果会通过ScannerResult/ScannerSymResult回调通知回来。

示例代码：

```
XcBarcodeScanner.init(this, new ScannerResult() {  
    @Override  
    public void onResult(String result) {  
        runOnUiThread(new Runnable() {  
            @Override  
            public void run() {  
                allResult = allResult + "\n" + result;  
                mTextResult.setText(allResult);  
                scrollToBottom();  
            }  
        });  
    }  
});  
  
XcBarcodeScanner.init(this, new ScannersSymResult() {  
    @Override  
    public void onResult(String sym, String barCode) {  
        runOnUiThread(new Runnable() {  
            @Override  
            public void run() {  
                allResult = allResult + "\n" + sym + ":" + barCode;  
                mTextResult.setText(allResult);  
                scrollToBottom();  
            }  
        });  
    }  
});
```

SDK反初始化

在Activity消亡或切换到后台时，需要进行反初始化动作，避免再次初始化后回调异常。

示例代码：

```
@Override
protected void onPause() {
    super.onPause();

    XcBarcodeScanner.deInit(this);
}
```

查询授权状态

使用如下接口可以查询授权状态：

```
XcBarcodeScanner.getLicenseState();
```

所有的授权状态定义请参考类：*LicenseState*

```
public class LicenseState {
    public static final int INACTIVE = 0;    // License not activated.
    public static final int ACTIVATING = 1; // License is activating.
    public static final int ACTIVED = 2;     // License already activated.
    public static final int INVALID = 3;     // License invalid, contact
    vendor.
    public static final int NETWORK_ISSUE = 4; // Network issue, connect
    before active license.
    public static final int EXPIRED = 5;     // License expired, contact
    vendor.
}
```

激活授权

使用如下接口可以按需要激活授权。

```
XcBarcodeScanner.activateLicense();
```

在调用激活授权后，需要约1~2分钟时间完成激活流程。
我们可以通过接口 *getLicenseState* 来查询授权的状态变化。

Note: 激活授权需要有良好的网络连接。

重新激活授权

使用如下接口可以清除缓存重新激活授权。

```
XcBarcodeScanner.forceNetworkActivate();
```

在调用激活授权后，需要约1~2分钟时间完成激活流程。
我们可以通过接口 *getLicenseState* 来查询授权的状态变化。

Note: 激活授权需要有良好的网络连接。

开始/结束扫码动作

可以通过SDK提供的接口控制扫码服务触发扫码动作的开始和结束。

```
XcBarcodeScanner.startScan();
XcBarcodeScanner.stopScan();
```

暂停/继续扫码服务

可以通过SDK提供的接口控制扫码服务的暂停和继续。

```
XcBarcodeScanner.suspendScanService();
XcBarcodeScanner.resumeScanService();
```

获取扫码服务的挂起状态

可以使用如下接口获取当前扫码服务的挂起状态。

```
boolean isScanServiceSuspending();
```

获取扫码服务版本号

可以通过SDK提供的接口获取扫码服务版本信息。

```
String serviceVer = XcBarcodeScanner.getServiceVersion();
Log.d(TAG, "Service ver: " + serviceVer);
```

获取SDK版本号

可以通过SDK提供的接口获取SDK自己的版本信息。

```
String sdkVer = XcBarcodeScanner.getSdkVersion();
Log.d(TAG, "SDK ver: " + sdkVer);
```

进阶设置

开关条码类型支持

可以通过SDK提供的接口控制扫码服务开启或关闭制定的条码类型。

```
XcBarcodeScanner.enableBarcodeType(String barcodeType, boolean enabled);
```

所有扫码类型都定义在BarcodeType类中

```
public class BarcodeType {
    public static final String AZTEC = "Aztec";
```

```

public static final String CODE11 = "Code11";
public static final String CODE39 = "Code39";
public static final String CODE93 = "Code93";
public static final String CODE128 = "Code128";
public static final String CODABAR = "Codabar";
public static final String DOTCODE = "Dotcode";
public static final String CODABLOCKF = "CODABLOCK F";
public static final String DATAMATRIX = "DATAMATRIX";
public static final String GS1_DATAMATRIX = "GS1DATAMATRIX";
public static final String EAN8 = "EAN-8";
public static final String EAN13 = "EAN-13";
public static final String GS1_DATABAR = "GS1_DATABAR";
public static final String HANXIN = "HANXIN";
public static final String HK25 = "HK25";
public static final String ITF25 = "ITF25";
public static final String MATRIX25 = "MATRIX25";
public static final String MAXICODE = "MAXICODE";
public static final String MSI = "MSI";
public static final String MICROPDF = "MICROPDF";
public static final String PDF417 = "PDF417";
public static final String USPS4ST = "USPS4ST";
public static final String QRCODE = "QRCODE";
public static final String INDUSTRIAL25 = "INDUSTRIAL25";
public static final String TELEPEN = "TELEPEN";
public static final String UPCA = "UPC-A";
public static final String UPCE = "UPC-E";
public static final String GS1_128 = "GS1-128";
public static final String GS1_DATABAR_LIMITED = "GS1_DataBarLimited";
public static final String GS1_DATABAR_EXPANDED = "GS1_DataBarExpanded";
public static final String IATA25 = "IATA25";
public static final String GRIDMATRIX = "Grid_Matrix";
}

```

示例代码：

```

XcBarcodeScanner.enableBarcodeType(BarcodeType.QRCODE, true); // 开启QRCode支持
XcBarcodeScanner.enableBarcodeType(BarcodeType.QRCODE, false); // 关闭QRCode支持

```

查询指定条码类型当前是否开启

所有扫码类型都定义在BarcodeType类中。

```
boolean isBarcodeTypeEnabled(String type)
```

输出方式设置

可以通过SDK提供的接口控制扫码服务输出扫码结果的方式。

```
XcBarcodeScanner.setOutputMethod(String method);
```

所有输出方式都定义在OutputMethod类中

```

public class OutputMethod {
    public static final String NONE = "NONE"; // No putput, application can
    get barcode data via callback.
    public static final String BROADCAST = "BROADCAST_EVENT"; // Output
    barcode data via broadcast.
    public static final String KEYBOARD = "KEYBOARD_EVENT"; // Output barcode
    data via keyboard simulate.
    public static final String CLIPBOARD = "CLIPBOARD_EVENT"; // Output
    barcode data via clipboard simulate.
    public static final String BROADCAST_KEYBOARD =
    "BROADCAST_EVENT/KEYBOARD_EVENT"; // Output barcode data via broadcast and
    keyboard simulate.
    public static final String BROADCAST_CLIPBOARD =
    "BROADCAST_EVENT/CLIPBOARD_EVENT"; // Output barcode data via broadcast and
    clipboard simulate.
}

```

示例代码：

```
XcBarcodeScanner.setOutputMethod(OutputMethod.CLIPBOARD);
```

扫码提示设置

可以通过SDK提供的接口设置扫码成功提示。

```
XcBarcodeScanner.setSuccessNotification(String notification);
```

所有提示类型都定义在NotificationType类中

```

public class NotificationType {
    public static final String MUTE = "Mute";
    public static final String SOUND = "Sound";
    public static final String VIBRATOR = "Vib";
    public static final String SOUND_VIBRATOR = "Sound/Vib";
}

```

示例代码：

```
XcBarcodeScanner.setSuccessNotification(NotificationType.SOUND);
```

扫码指示灯设置

可以通过SDK提供的接口设置扫码成功指示灯功能开关。

```
XcBarcodeScanner.enableSuccessIndicator(boolean enabled);
```

示例代码：

```

XcBarcodeScanner.enableSuccessIndicator(true); // 开启扫码指示灯功能
XcBarcodeScanner.enableSuccessIndicator(false); // 关闭扫码指示灯功能

```

扫码超时设置

可以通过SDK提供的接口设置扫码超时时间。

```
XcBarcodeScanner.setTimeout(int seconds);
```

超时时间范围为1~9秒。

示例代码：

```
XcBarcodeScanner.setTimeout(5); // 设置超时时间为5秒
```

瞄准灯工作模式设置

可以通过SDK提供的接口设置瞄准灯的工作模式。

```
void setAimerLightsMode(int aimMode);
```

瞄准灯支持的工作模式在AimerMode类中定义：

```
public class AimerMode {  
    public static int ALWAYS_OFF; // Always off.  
    public static int TRIGGER_ON; // TurnOn while scanning.  
    public static int ALWAYS_ON; // Always on.  
}
```

示例代码：

```
XcBarcodeScanner.setAimerLightsMode(AimerMode.TRIGGER_ON); // 设置瞄准灯工作模式  
为：扫码时亮起  
XcBarcodeScanner.setAimerLightsMode(AimerMode.ALWAYS_OFF); // 设置瞄准灯工作模式  
为：一直关闭
```

注意：如果激光瞄准灯，请勿使用常亮（ALWAYS_ON）工作模式。

补光灯工作模式设置

可以通过SDK提供的接口设置补光灯的工作模式。

```
void setFlashLightsMode(int flashMode);
```

补光灯支持的工作模式在FlashMode类中定义：

```
public class FlashMode {  
    public static int OFF; // Always off.  
    public static int ILLUME_ONLY; // Only illume.  
    public static int ILLUME_STROBE; // Illume and Strobe.  
}
```

示例代码：

```
XcBarcodeScanner.setFlashLightsMode(FlashMode.ILLUME_STROBE); // 设置补光灯工作  
模式为：补光+闪光  
XcBarcodeScanner.setFlashLightsMode(FlashMode.ILLUME_ONLY); // 设置补光灯工作  
模式为：仅补光
```

注意：补光灯工作时，仅会在扫码时点亮，闪光（STROBE）模式会在图像传感器曝光的瞬间输出高亮光。

扫码结果大小写设置

可以通过SDK提供的接口设置扫码结果的大小写转换选项。

```
XcBarcodeScanner.setTextCase(String textCase);
```

所有转换选项都定义在TextCaseType类中

```
public class TextCaseType {  
    public static final String NONE = "NONE_CASE";  
    public static final String UPPER = "UPPER_CASE";  
    public static final String LOWER = "LOWER_CASE";  
}
```

示例代码：

```
XcBarcodeScanner.setTextCase(TextCaseType.NONE); // 设置扫码结果大小写不转换  
XcBarcodeScanner.setTextCase(TextCaseType.UPPER); // 设置扫码结果转换为大写  
XcBarcodeScanner.setTextCase(TextCaseType.LOWER); // 设置扫码结果转换为小写
```

扫码结果前缀设置

可以通过SDK提供的接口为扫码结果添加指定的前缀。

```
XcBarcodeScanner.setTextPrefix(String prefix);  
XcBarcodeScanner.setTextPrefix2(String prefix2);
```

示例代码：

```
XcBarcodeScanner.setTextPrefix("<"); // 设置扫码结果前缀为“<”  
XcBarcodeScanner.setTextPrefix2(":"); // 设置扫码结果前缀2为“:”  
XcBarcodeScanner.setTextPrefix("Empty"); // 设置扫码结果前缀为空
```

扫码结果后缀设置

可以通过SDK提供的接口为扫码结果添加指定的后缀。

```
XcBarcodeScanner.setTextSuffix(String prefix);  
XcBarcodeScanner.setTextSuffix1(String prefix2);
```

示例代码：

```
XcBarcodeScanner.setTextSuffix(">"); // 设置扫码结果后缀为“>”
XcBarcodeScanner.setTextSuffix1(":"); // 设置扫码结果后缀2为“:”
XcBarcodeScanner.setTextSuffix("Empty"); // 设置扫码结果后缀为空
```

设置连续扫码间隔时间

可以通过SDK提供的接口设置连续扫码的间隔时间。

```
setLoopScanInterval(int ms);
```

示例代码：

```
XcBarcodeScanner.setLoopScanInterval(100); // 设置连续扫码间隔时间为100毫秒
```

获取连续扫码状态

可以通过SDK提供的接口获取当前是否正在进行连续扫码。

```
boolean isLoopScanRunning();
```

开始/停止连续扫码

可以通过SDK提供的接口触发连续扫码。

```
startLoopScan();
stopLoopScan();
```

示例代码：

```
// 开始连续扫码
XcBarcodeScanner.setLoopScanInterval(100); // 设置连续扫码间隔时间为100毫秒
XcBarcodeScanner.startLoopScan(); // 开始连续扫码

// 结束连续扫码
if (XcBarcodeScanner.isLoopScanRunning()) { // 检查当前是否正在进行连续扫码
    XcBarcodeScanner.stopLoopScan(); // 停止连续扫码
}
```

设置连续扫码

可以通过SDK提供的接口设置连续扫码模式（不自动触发扫码动作）。

```
void enableLoopModeNotScan(boolean isEnabled);
```

示例代码：

```
// 设置为连续扫码
XcBarcodeScanner.enableLoopModeNotScan(true); // 仅开启连续扫码模式，不自动触发
```


获取多条码开启情况

可以通过SDK提供的接口获取当前是否开启了多条码功能。

```
boolean isSupportMultiBarcodes();
```

开启/关闭多条码功能

可以通过SDK提供的接口开启或者关闭多条码功能。

```
void enableMultiBarcodes(boolean isEnabled);
```

示例代码：

```
//开启多条码功能
if(!XcBarcodeScanner.isSupportMultiBarcodes()){//检测多条码功能是否开启
    XcBarcodeScanner.enableMultiBarcodes(true);//开启多条码功能
}
```

设置多条码参数

可以通过SDK提供的接口，设置多条码扫描参数。

```
void setMultiBarcodes(int numberOfBarcodes, boolean fixedNumber);
```

参数说明：

numberOfBarcodes - 最大单次扫描的条码数量，范围：1~20
fixedNumber - 是否固定条码数量，**true**表示固定条码数量，**false**表示非固定条码数量。

注：固定条码数量是指必须达到指定数量的条码才能解码成功；非固定条码数量则是解出了几个条码就输出几个条码。

示例代码：

```
// 设置多条码数量为3，且固定条码数量
XcBarcodeScanner.setMultiBarcodes(3, true);

// 设置多条码数量为1（每次只尝试解出1个条码）
XcBarcodeScanner.setMultiBarcodes(1, false);
```

注：当多条码数量设置为1时，就是默认的单条码模式，此时固定条码数量的设置值无实际意义。

设置条码识别范围

可以通过SDK提供的接口，设置条码识别的范围，典型应用场景为一维码精准扫描。

```
void setScanRegionSize(int regionSize);
```

支持的regionSize定义在RegionSizeType类中：

```

public class RegionSizeType {
    public static final int VIEWSIZE_100 = 0; // 100% 画幅
    public static final int VIEWSIZE_75 = 1; // 75% 画幅
    public static final int VIEWSIZE_50 = 2; // 50% 画幅
    public static final int VIEWSIZE_25 = 3; // 25% 画幅
    public static final int VIEWSIZE_12 = 4; // 12% 画幅
    public static final int VIEWSIZE_6 = 5; // 6% 画幅
    public static final int VIEWSIZE_3 = 6; // 3% 画幅
    public static final int VIEWSIZE_1 = 7; // 1% 画幅
    public static final int VIEWSIZE_1D = 8; // 一维码专用画幅（图像的中间几行）
}

```

注：固定条码数量是指必须达到指定数量的条码才能解码成功；非固定条码数量则是解出了几个条码就输出几个条码。

示例代码：

```

// 设置条码识别精度为一维码精准识别
XcBarcodeScanner.setScanRegionSize(RegionSizeType.VIEWSIZE_1D);

// 设置条码识别精度为最低（整个100%画面范围内的条码都识别）
XcBarcodeScanner.setScanRegionSize(RegionSizeType.VIEWSIZE_100);

```

注：如果开启了多条码识别，推荐使用100%画幅的识别范围。

获取最后一张解码图片

可以通过以下接口获取最后一张送去解码的图片。

```

XCImage getLastDecodeImage();

```

XCImage 是返回的图片对象。

示例代码：

```

import com.tools.XCImage;

public void getLastImage() {
    XCImage lastImg=XcBarcodeScanner.getLastDecodeImage();

    if(lastImg!=null){
        String infoStr="width: "+lastImg.getWidth()+" , Height: "+lastImg.getHeight()+" , Stride: "+lastImg.getStride()+" , size: "+lastImg.getData().length+" Bytes";
        showAlertDialog("Image Info:",infoStr,false,"OK",null);
    }else{
        showAlertDialog("Image Info:","No image!",false,"OK",null);
    }
}

```

设置自定义广播

将自定义广播的Action和接受扫码结果的Key通过该接口配置好后，就可以通过广播接收到扫码结果了。

```
void setScanResultBroadcast(String action, String resultKey);
```

使用示例：

```
XcBarcodeScanner.setScanResultBroadcast("xxx.Action", "scanResultKey");
```

禁用/启用扫码快捷按键

根据设备设计不同，目前所支持的快捷键有：左侧边扫码键/右侧边扫码键/正面扫码键/手持柄扫码键。针对不同的快捷键，提供了对应的禁用/启用扫码功能接口。禁用后，按下扫码按键将无法扫码；启用后，按键恢复扫码功能。

```
void setFrontScanKeyEnable(boolean isEnabled); //正面扫码按键  
  
void setLeftScanKeyEnable(boolean isEnabled); //左侧边扫码键  
  
void setRightScanKeyEnable(boolean isEnabled); //右侧边扫码键  
  
void setPoGoScanKeyEnable(boolean isEnabled); //手持柄扫码键
```

使用示例：

```
XcBarcodeScanner.setFrontScanKeyEnable(false); //禁用正面扫码按键扫码功能  
  
XcBarcodeScanner.setFrontScanKeyEnable(true); //启用正面扫码按键扫码功能
```

配置自定义扫码键

可以通过SDK提供的接口设置自定义扫码按键。

```
boolean setCustomScanKey(int keyCode);
```

使用示例：

```
XcBarcodeScanner.setCustomScanKey(32); //设置KEYCODE_D为扫码键
```

注意：KEYCODE_DEL、KEYCODE_POWER、KEYCODE_CLEAR、KEYCODE_BACK、KEYCODE_HOME、KEYCODE_GRAVE不可设置为扫码键

配置管理

导入、导出或者重置扫码配置

```
void loadSettings(); //导入设置  
void saveSettings(); //导出设置  
void resetSettings(); //重置设置
```

DataMatrix条码配置

可以通过SDK提供的接口对DataMatrix进行设置。

```
void setDataMatrixWithSeparators(boolean withSeparators); //设置显示分隔符，true显示 false 不显示
void setDataMatrixMaxOutputLength(int maxLength); //设置最大输出长度（0:不限制）
```

示例代码：

```
XcBarcodeScanner.setDataMatrixWithSeparators(true); //开启DataMatrix的分隔符显示

XcBarcodeScanner.setDataMatrixMaxOutputLength(2); // 设置DataMatrix解码的最大输出长度为2
```

设置扫码失败提示模式

可以通过SDK提供的接口设置扫码失败提示模式。

```
void setFailedNotification(String notification);
```

notification支持的参数和含义：

"Mute"	-----无
"Sound"	-----声音提示
"vib"	-----振动提示
"Sound/Vib"	-----声音和振动提示

示例代码：

```
// 设置扫码失败有声音提示
XcBarcodeScanner.setFailedNotification("Sound");
```

获取扫码功能状态

可以通过SDK提供的接口获取扫码功能是否开启。

```
boolean isScanEnable(); //获取当前扫码功能是否可用
```

禁用/开启扫码功能

可以通过SDK提供的接口禁用扫码和开启功能。

```
void enableScan(boolean isEnabled); //禁用/开启扫码功能
```

使用示例：

```
if(XcBarcodeScanner.isScanEnable()){//如果获取到扫码功能已开启  
    xcBarcodeScanner.enableScan(false); //禁用扫码功能  
}
```